



Internship Report

Prediction of Water Distribution Network Flow via Inference Methods on Graph Nodes with Missing Data

Algassimou DIALLO

2024-2025

LJAD internship supervisors:
Rémi CATELLIER
Yassine LAGUEL

Msc DSAI supervisor:
Michel RIVEIL

Contents

1	Introduction	2
2	Dataset and Preprocessing	3
2.1	Missing Data	3
2.2	Preprocessing Steps	4
2.3	Final Missing Data Analysis	4
3	Imputation Strategy and Evaluation Framework	5
3.1	Controlled Masking Strategy	5
3.2	Cross-Validation Procedure	6
3.3	Evaluation Datasets	6
4	Imputation Methods	6
4.1	Prophet	6
4.2	BRITS: Bidirectional Recurrent Imputation for Time Series	7
4.3	SAITS: Self-Attention-based Imputation for Time Series	8
4.4	PyPOTS	9
4.5	Preliminary Evaluation Results	9
4.6	Final Imputation Results	10
4.7	Final Imputation Strategy	12
5	Graph-Based Inference on the Water Distribution Network	14
5.1	Introduction and Objectives	14
5.2	Dumora’s Thesis overview	14
5.3	Reconstruction of the Water Distribution Network as a Graph	14
5.4	Reimplementation of the Flow-Based Inference Algorithm for Hydraulic Reconstruction of a Water Distribution Network	15
5.4.1	Transport Network and Basic Ford-Fulkerson Algorithm	15
5.4.2	Challenges of Water Distribution Networks	15
5.4.3	Graph Adaptation Following Dumora’s Methodology	15
5.5	Constrained Ford-Fulkerson Inference Method	16
5.6	Implementation and Validation	17
5.7	Limitations and Issues with Available Data	19
5.8	Results of Constrained Ford-Fulkerson method on the subgraph	20
6	Kernel Ridge Regression	21
6.1	Kernel Ridge Regression on Graphs	21
6.2	Results and Limitations of the Method	23
7	Redefining the Regression Problem to Enforce Kirchhoff’s Law	24
8	Conclusion	25

1 Introduction

This internship is conducted at the CNRS - Laboratoire J.A. Dieudonné (LJAD), within the Probability and Statistics team, as part of a collaborative project between CNRS, the Université Côte d’Azur (UCA), and the public water utility company Eau d’Azur. The work is supervised by Rémi Catellier and Yassine Laguel (CNRS/LJAD) and Agathe Maupetit (Eau d’Azur).

The goal of the project is the *prediction of water distribution network flow through inference methods on graph nodes with missing data*. Water distribution networks are instrumented with sensors that measure flow and pressure at various points. However, due to the heterogeneous deployment of these sensors, the available data exhibits variable temporal resolution and frequent missing values. The project aims to leverage both statistical modeling and the structural information of the network to improve data quality and enable accurate flow predictions.

The objectives of the internship were:

- To study and implement methods for imputing missing data in the time series recorded by the sensors.
- To represent the water distribution network as a graph and exploit its geometry to propagate information between connected nodes.
- To implement and adapt methods proposed in the PhD thesis [1] to perform flow inference across the network.

I will first present the dataset provided by Eau d’Azur, including its structure, the challenges posed by missing values, and the preprocessing steps required for further analysis. This is followed by a detailed study of missing data imputation methods, starting from the reproduction of previous approaches and leading to the development of our hybrid imputation method for the missing values. In the final part of the report, I describe the construction of the water distribution network graph and the implementation of inference methods inspired by [1], aimed at predicting flow across the network.

2 Dataset and Preprocessing

The data used in this project is provided by Eau d'Azur and corresponds to the potable water distribution network of the commune of La Trinité, located in the Côte d'Azur region. Figure 1 illustrates the structure of the network: blue lines represent water pipes (*conduites d'eau*), and green-labeled circles correspond to various metering points (*compteurs*) across the network.

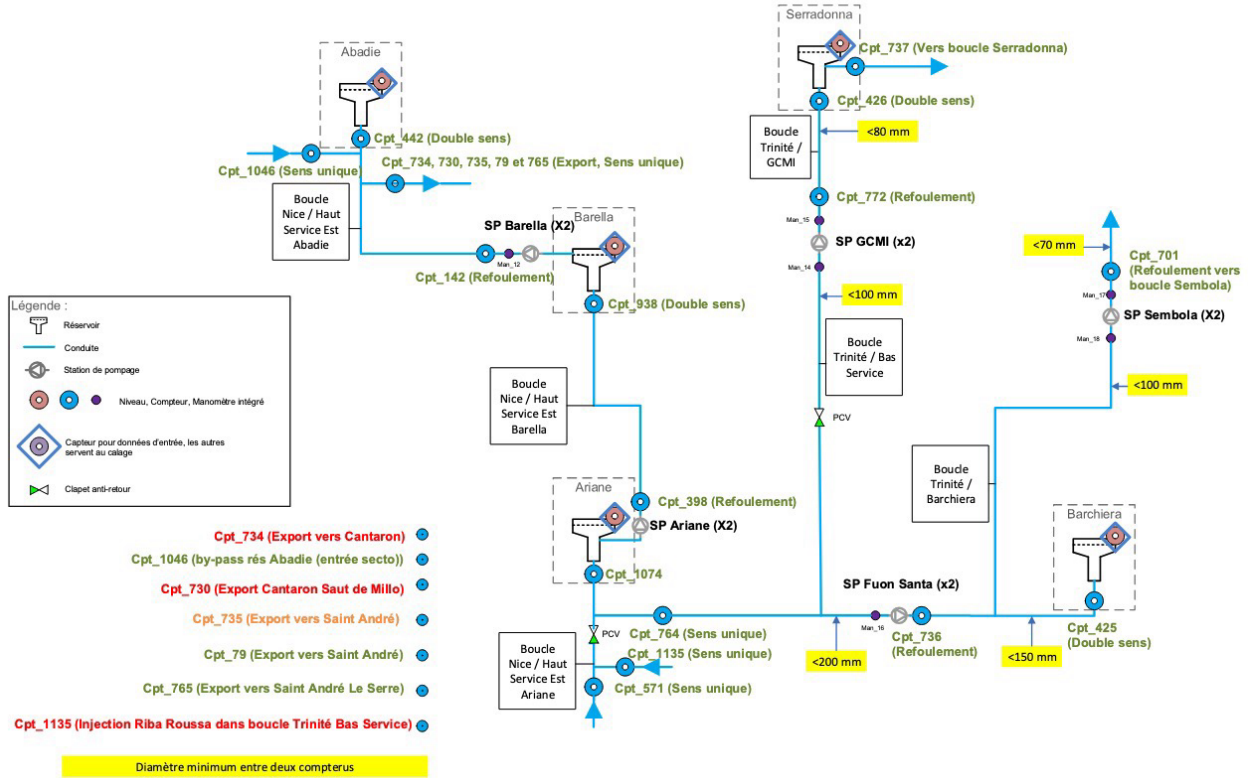


Figure 1: Network of the commune La Trinité

The dataset consists of multiple time series recorded at various metering points, including measurements such as water flow rates (débits), volumes, and binary signals (valeurs TOR) indicating the operational status (on/off) of components like pumps. All time series are recorded at a fixed 15-minute interval.

2.1 Missing Data

In previous internship [2], preprocessing was applied to the sensor data. However, the resulting missing data rates were extremely high, with some sensors showing over 80% missing values (Table 1). I analyzed the preprocessing pipeline in detail and found several inconsistencies, including improper handling of TOR signals, incorrect thresholding, and potential errors in time series alignment. These mistakes likely caused a significant overestimation of missing data.

Given these findings, we decided to reconstruct the preprocessing pipeline from the raw dataset.

Table 1: Missing values reported in previous internship[2]

Sensor	Missing Data (%)
BARCHIERA_DIST_DEB	24
BARCHIERA_ENT_DEB	83
FUONSANT_REF_DEB	78
GCMI_REF_DEB	82
SEMBOLA_REF_DEB	96
SERRADONNA_DIST_DEB	40
SERRADONNA_ENT_DEB	83

2.2 Preprocessing Steps

I applied several preprocessing operations to improve data consistency and ensure accurate missing data analysis:

I merged the distribution (DIST) and entry (ENT) flow time series for Barchiera and Serradonna by adding the ENT series values with a negative sign to preserve the flow directionality within the combined signal. I converted the TOR columns, which indicate pump activation status, to a binary format where 0 means OFF and 1 means ON. I applied domain-specific physical thresholds, defined through technical discussions with Eau d’Azur, to remove invalid values: I limited pump measurements to the range $(-2, 2)$, reservoir levels to $(0, 100)$, main meter (CPT_764) readings to $(0, 100,000)$, and standard flow meters (“autres débits”) to $(0, 1,000)$. I also used the TOR signals to correct flow data by setting values to NaN whenever a pump was OFF but a non-zero flow was recorded.

2.3 Final Missing Data Analysis

After applying the revised preprocessing, we obtain the following missing data proportion (Table 2).

Table 2: Final proportion of missing values per sensor

Sensor	Missing Data (%)
BARCHIERA_DEB	3.76
FUONSANT_REF_DEB	0.20
SERRADONNA_DEB	22.05
GCMI_REF_DEB	0.10
SEMBOLA_REF_DEB	0.57

Compared to the previous results, the missing data rates were significantly reduced. Sensors such as FUONSANT, GCMI, and SEMBOLA, which were previously reported with missing rates exceeding 78%, now show near-complete data availability.

The raw data consists of time series with a native 15-minute interval. However, some entire days are missing, which distorts the estimation of missing data rates. To correct this, I reconstructed the complete timestamp grid by inserting 96 missing points for each absent day, since a full day equals 96 intervals of 15 minutes. This adjustment provides an accurate measure of missing data percentages. For instance, after this correction, the Barchiera site exhibited a missing data rate of approximately 13%.

It is important to note that this re-alignment is only for assessing data completeness. For the actual modeling and machine learning tasks, we retained the original time series without artificially inserting missing timestamps. Forecasting models like Prophet [3] inherently handle irregular time series and missing dates due to their piecewise linear or saturating trend components, making explicit NaN-filling unnecessary. [3]

Another important observation from the data analysis phase is that missing values do not occur independently or randomly. Instead, they tend to appear in blocks. For instance, in the Barchiera time series, the

longest continuous block of missing values spans 623 consecutive time steps which corresponds to approximately 6.5 days. (see Figure 2). This behavior has implications for the imputation process, as models need to handle both isolated gaps and extended periods of missing data.

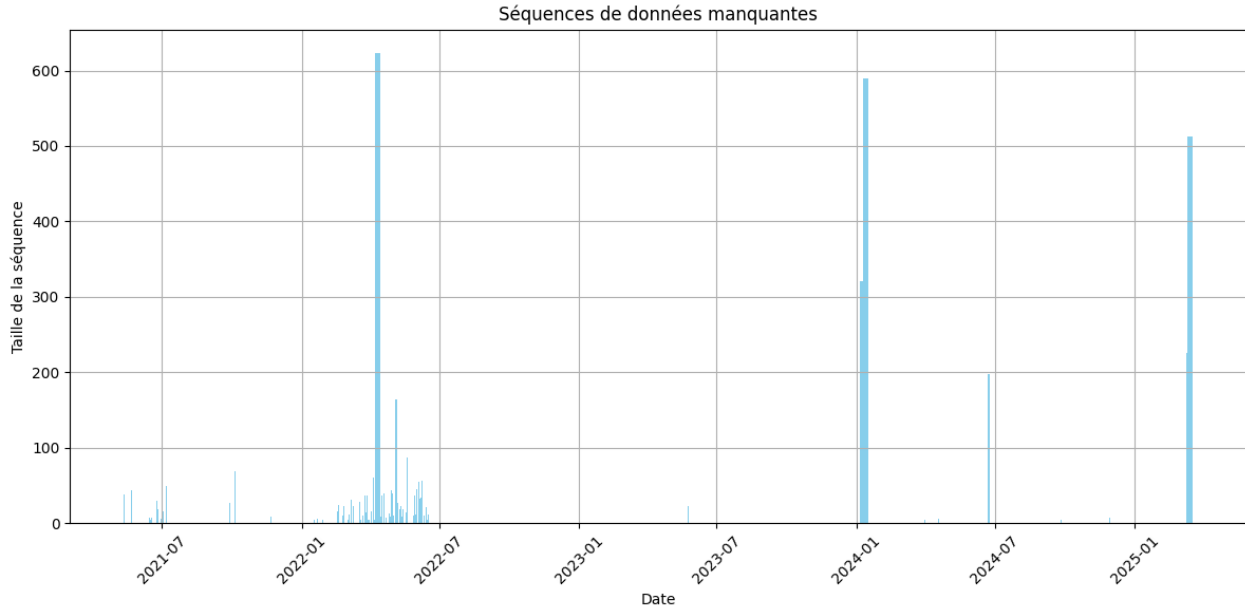


Figure 2: Missing data blocks in Barchiera time-series

3 Imputation Strategy and Evaluation Framework

Given the quantity and structure of missing data, the next objective is to develop a systematic approach for testing imputation methods capable of capturing temporal patterns, such as seasonality. To that end, I implemented a controlled masking framework to evaluate different imputation models under realistic conditions.

3.1 Controlled Masking Strategy

When working with real-world data already containing missing values, evaluating imputation accuracy directly is not possible. To address this, I applied controlled masking strategies to known (non-missing) values. These artificially masked values provide ground truth for assessing imputation performance.

I used two masking strategies to evaluate the robustness of imputation methods:

- **Random masking:** A fixed proportion of observed (non-NaN) values is randomly selected and replaced with NaNs. We test with two masking rates: 5% masking, and 15% masking, reflecting the missing data rate observed at the Barchiera series.
- **Block masking:** Several sequences of observed values are replaced with NaNs to simulate realistic missing data blocks. The block sizes are chosen to match the distribution of missing sequences found in the Barchiera dataset (Figure 2), specifically using block lengths from the set:

$$\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 15, 20, 30, 40, 50, 70, 100, 200, 300, 500\}$$

The masked datasets were used to compare the performance of different imputation methods (e.g., Prophet [3], linear interpolation, KNN imputer [4]) using standard error metrics:

- **Mean Absolute Error (MAE):**

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- **Mean Squared Error (MSE):**

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where y_i are the true values and $\hat{y}_i$ are the imputed estimates at position i .

3.2 Cross-Validation Procedure

To robustly evaluate the imputation methods, I implemented a cross-validation framework based on the block sizes list defined in the masking strategy. For each block size, I generate 100 masked datasets by randomly placing missing blocks of the given length in the observed data. Each imputation method is then applied to all 100 datasets, and the Mean Squared Error (MSE) was computed for each run.

The average MSE over the 100 runs for each block size is used as a performance metric to assess model accuracy across different missing data patterns and lengths.

3.3 Evaluation Datasets

I evaluate the imputation framework on the following datasets:

- **Barchiera time series:** A subset without missing values, extracted from continuous and complete segments, is used to simulate controlled missing data scenarios through masking strategies.
- **External benchmark datasets:** The *Electricity* dataset [5] and the *Beijing Multi-Site Air Quality* dataset [6], both from the UCI Machine Learning Repository.

4 Imputation Methods

I conducted a literature review and consulted several survey papers on time series imputation to identify the most commonly used and effective methods.[7][8][9] Based on this research and the characteristics of our dataset, we selected a set of baseline and advanced imputation techniques for the internship.

We applied baseline methods including linear interpolation, spline interpolation, Kalam Filters [10] mean and median imputation, and the K-Nearest Neighbors (KNN) imputer [4]. In addition, we implemented two state-of-the-art deep learning models that have shown strong benchmark performance [8]: BRITS (Bidirectional Recurrent Imputation for Time Series) [11] and SAITS (Self-Attention-based Imputation for Time Series) [12]. We also used the Prophet[3] model developed by Facebook.

4.1 Prophet

Prophet [3] models a time series as a sum of trend, seasonality, holiday effects, and noise:

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t,$$

where $g(t)$ captures non-periodic changes in the trend, $s(t)$ models periodic seasonal patterns such as weekly or yearly effects, $h(t)$ represents holiday effects occurring on irregular dates, and ϵ_t is an error term assumed to be normally distributed: $\epsilon_t \sim \mathcal{N}(0, \sigma^2)$.

Prophet [3] includes two main trend models: a saturating growth model and a piecewise linear model.

The saturating growth model uses a logistic function to represent growth with a carrying capacity C :

$$g(t) = \frac{C}{1 + \exp(-k(t - m))},$$

where k is the growth rate and m is an offset parameter.

Prophet [3] also uses a piecewise linear model with changepoints. This model allows the growth rate to change at predefined points in time. The trend function is given by:

$$g(t) = (k + a(t)^\top \delta) t + (m + a(t)^\top \gamma),$$

where k is the base growth rate, δ contains rate adjustments at changepoints, m is the initial offset, and γ_j is set to $-s_j \delta_j$ to ensure continuity of the trend at each changepoint s_j .

4.2 BRITS: Bidirectional Recurrent Imputation for Time Series

BRITS [11] formulates missing value imputation as a recurrent dynamical system that learns to impute missing entries directly within a bidirectional recurrent neural network (RNN) framework. Unlike prior approaches that smooth missing values heuristically, BRITS [11] treats missing values as variables updated through recurrent dynamics based on observed data, which improves imputation accuracy and downstream prediction performance.

Initially, BRITS [11] models time series where variables observed at the same time step are assumed uncorrelated, applying unidirectional recurrent dynamics (RITS-I) [3]. At each time step t , if the observation x_t is present, it validates the imputation and feeds the true value forward. If missing, the model imputes $\hat{x}_t$ using the hidden state from previous steps, and the estimation error is computed retrospectively at future steps dependent on $\hat{x}_t$. This delayed error enables effective learning despite missing targets.

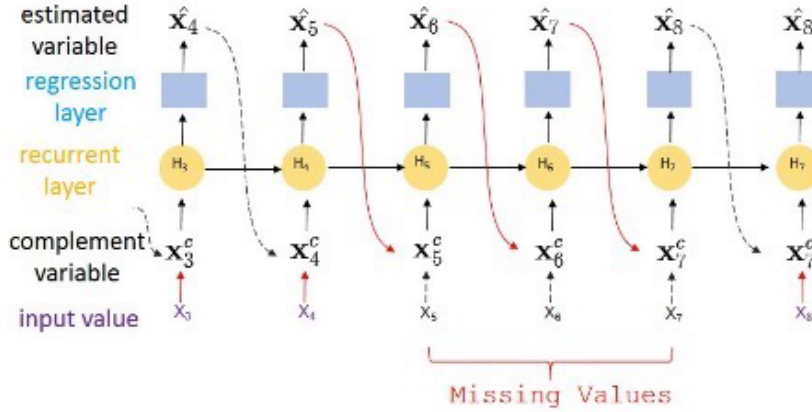


Figure 3: RITS-I architecture: Imputation with unidirectional dynamics. Taken from original paper[11]

The model combines two components: a recurrent component represented by an RNN that propagates hidden states and a regression component that estimates missing values from these states. Formally, the hidden state update is:

$$h_t = \sigma(W_h(h_{t-1} \odot \gamma_t) + U_h(x_t^c \odot m_t) + b_h)$$

where x_t^c is the complement input replacing missing values in x_t with imputed values $\hat{x}_t$, m_t is the mask indicating observed entries, and γ_t is a temporal decay factor accounting for irregular time intervals. The imputed value is generated as:

$$\hat{x}_t = W_x h_{t-1} + b_x$$

The temporal decay is defined as

$$\gamma_t = \exp(-\max(0, W_\gamma \delta_t + b_\gamma))$$

which models how information decays over time gaps δ_t between observations, crucial for irregularly sampled data.

BRITS [11] extends this to bidirectional recurrent dynamics by running the imputation process forwards and backwards in time, refining estimates with information from both past and future observations. Missing values are treated as learnable variables within the computational graph, updated iteratively via backpropagation through time in both directions.

4.3 SAITS: Self-Attention-based Imputation for Time Series

SAITS [12] is a Transformer-inspired model designed for multivariate time series imputation. It addresses the challenge of missing data by leveraging a self-attention mechanism tailored for imputation tasks. Unlike standard Transformers, SAITS [12] incorporates Diagonally-Masked Self-Attention (DMSA), which prevents a time step from attending to itself. This encourages the model to infer missing values based on correlations across other time steps and features.

The architecture consists of two DMSA blocks followed by a weighted combination module. Each DMSA block refines the feature representations through multi-head attention, positional encoding, and feed-forward layers, gradually improving the imputations. The first block produces an initial estimation by filling in missing values, while the second block further refines these estimations through a deeper network structure.

A weighted combination block dynamically fuses the outputs of the two DMSA blocks, balancing the contributions of coarse and refined imputations using attention-derived weights. This mechanism allows SAITS [12] to adaptively weigh information based on temporal dependencies and missingness patterns.

For training, SAITS [12] employs two loss components: a reconstruction loss (ORT) computed over intermediate imputations and a final imputation loss (MIT) computed over the model’s output.

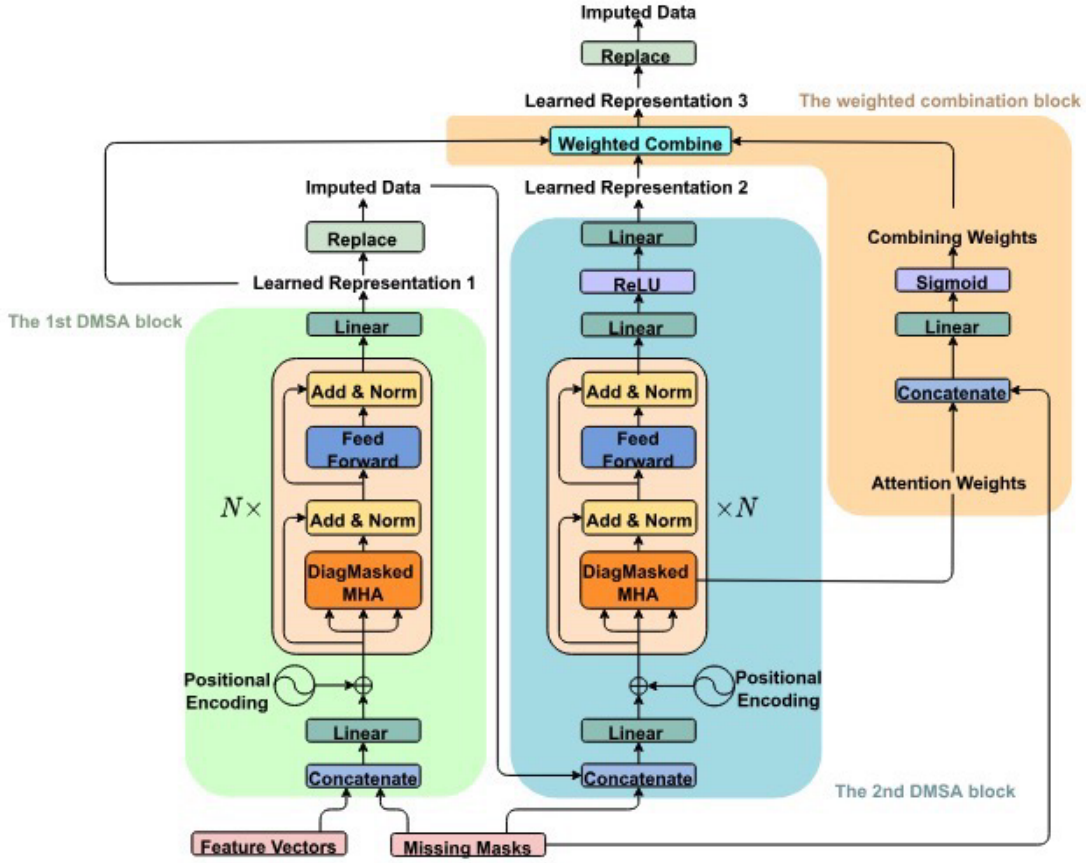


Figure 4: SAITS model architecture taken from original paper [12]

4.4 PyPOTS

PyPOTS [13] is a Python package that implements time series imputation models used in benchmark studies [8], including BRITS, SAITS [12], and other advanced architectures. It standardizes workflows for handling missing data patterns, model training, evaluation, and performance benchmarking.

I integrated PyPOTS into this project to evaluate BRITS and SAITS on our water network datasets.

4.5 Preliminary Evaluation Results

During the literature review, I found that simple methods like linear interpolation often outperform more complex models for time series imputation. Furthermore, the missing data pattern significantly affects model performance [8]. Our initial experiments confirms these observations.

Using our evaluation framework, we tested the imputations methods on the two benchmark datasets: Electricity, AirQuality, and our Barchiera time series. The results are reported in the tables 3 and 4.

Table 3: Imputation Results — Random Masking (5% missing) (Rescaled MSE / MAE)

Method	Electricity	AirQuality	Barchiera
Linear	0.99 / 0.74	0.51 / 0.44	2.03 / 2.10
Spline	1.32 / 0.85	1.24 / 0.58	2.53 / 2.75
Prophet	5.22 / 1.72	49.53 / 3.78	4.70 / 4.74
Mean	19.00 / 3.39	198.48 / 8.26	6.88 / 6.02
Median	19.26 / 3.34	208.44 / 8.25	7.45 / 4.96

Table 4: Imputation Results — Block Masking (Block size = 40, n = 3) (Rescaled MSE / MAE)

Method	Electricity	AirQuality	Barchiera
Prophet	4.79 / 1.53	39.09 / 3.02	4.10 / 5.22
Linear	5.48 / 1.67	1.87 / 0.66	10.09 / 6.10
Median	12.65 / 2.77	141.05 / 6.57	8.04 / 5.70
Mean	15.15 / 3.05	222.15 / 7.69	6.92 / 6.37
Spline	17.49 / 3.09	50.86 / 2.59	484.53 / 41.70

Our experiments shows that linear interpolation provides the best performance when imputing small amounts of randomly missing data. However, when handling block-missing scenarios, Prophet [3] outperforms other methods by maintaining low error rates.

To further investigate, I performed the cross-validation with linear and Pophet [3] over the different block sizes length and results shows that linear interpolation dominates for small block sizes (up to 15 points). Beyond this threshold, its error grows rapidly, while Prophet’s performance remains stable and overtakes linear interpolation as block size increases.(figure 5)

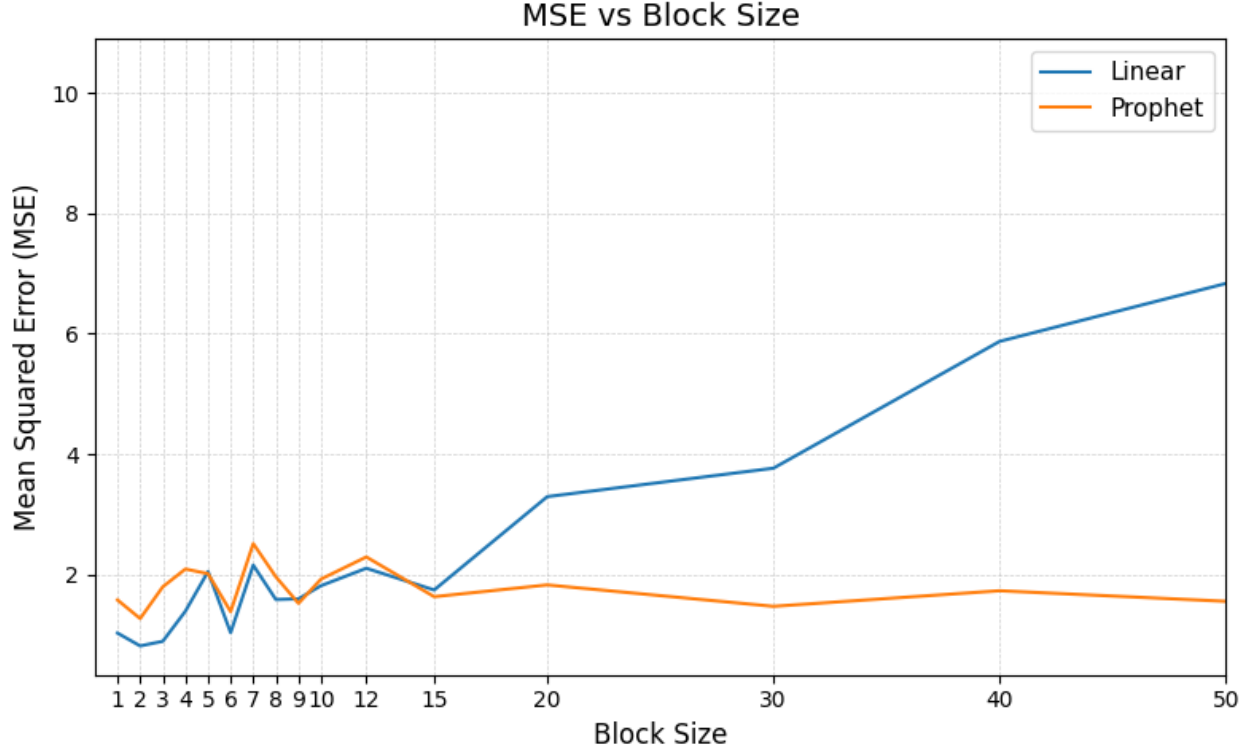


Figure 5: MSE performance of linear vs Prophet obtained via cross-validation

4.6 Final Imputation Results

After observing the strong performance of linear interpolation and Prophet [3] in our preliminary experiments, we extended the comparison to include two state-of-the-art deep learning models: BRITS and SAITS. To ensure a fair evaluation, we adapted our cross-validation strategy accordingly. Specifically, we first split the datasets into a training set (80%) and a test set (20%). We trained BRITS [11] and SAITS [12] exclusively on the training data. Subsequently, we applied the cross-validation solely on the test set, ensuring the evaluation remained unbiased.

Figure 8 summarizes the MSE performance of Linear, Prophet [3], BRITS [11], and SAITS [12] across various block sizes. As in previous experiments, linear interpolation consistently outperforms other methods for small block sizes. However, as the block size increases, its error increases sharply, becoming the worst-performing method for large gaps.

Prophet [3] maintains stable performance across all block sizes, consistently achieving the lowest error, outperforming both BRITS [11] and SAITS [12]. While BRITS [11] and SAITS [12] perform similarly and surpass linear interpolation on larger missing blocks, they do not reach Prophet’s [3] performance thus making linear the best method for imputing on small block sizes and Prophet [3] the best on larger block sizes.

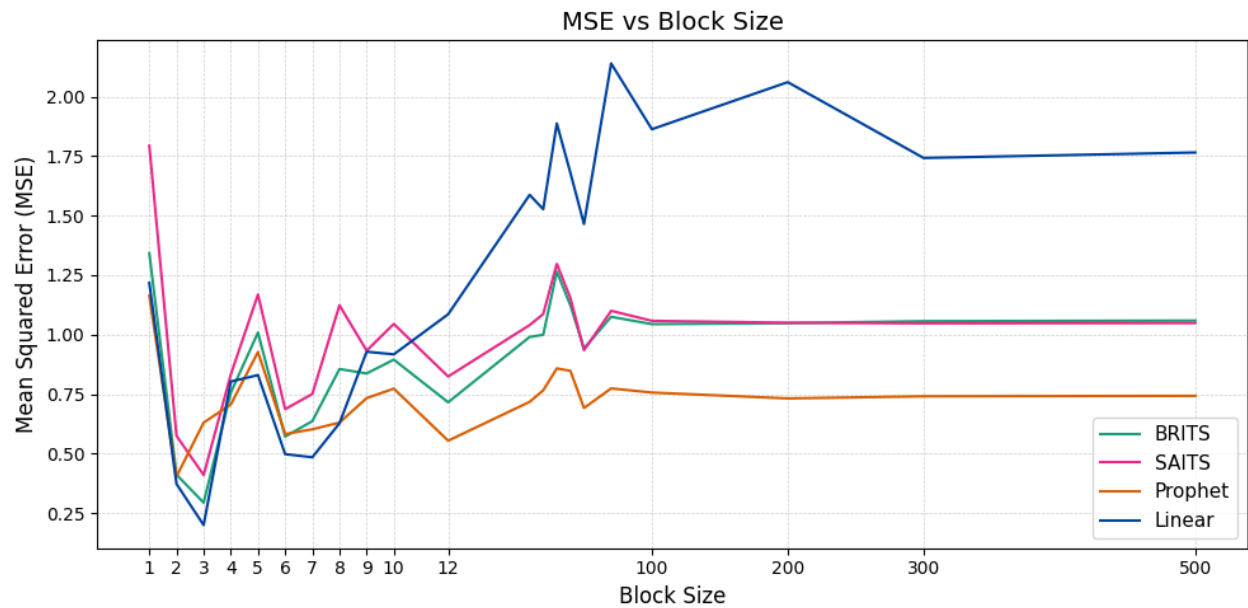


Figure 6: MSE performance of Linear, Prophet, BRITS, and SAITS across varying block sizes.

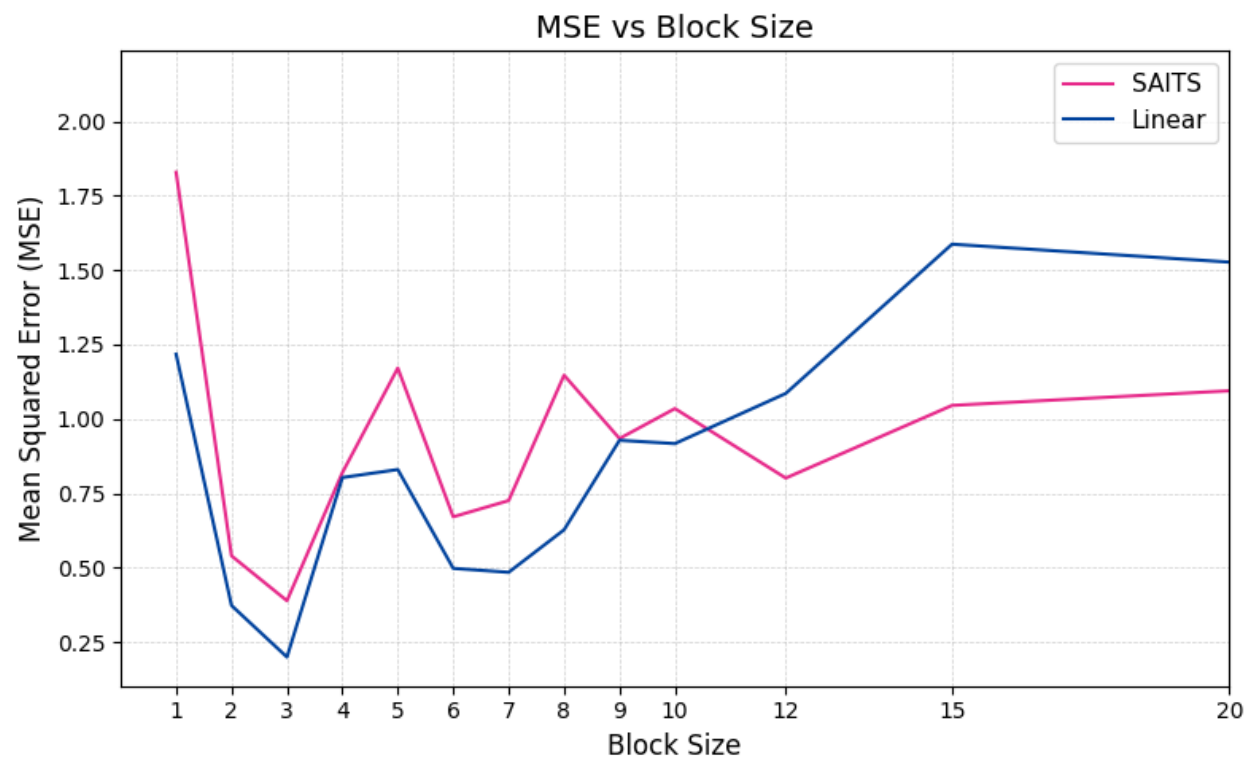


Figure 7: Zoom on Linear vs SAITS results from figure 6 for small block sizes

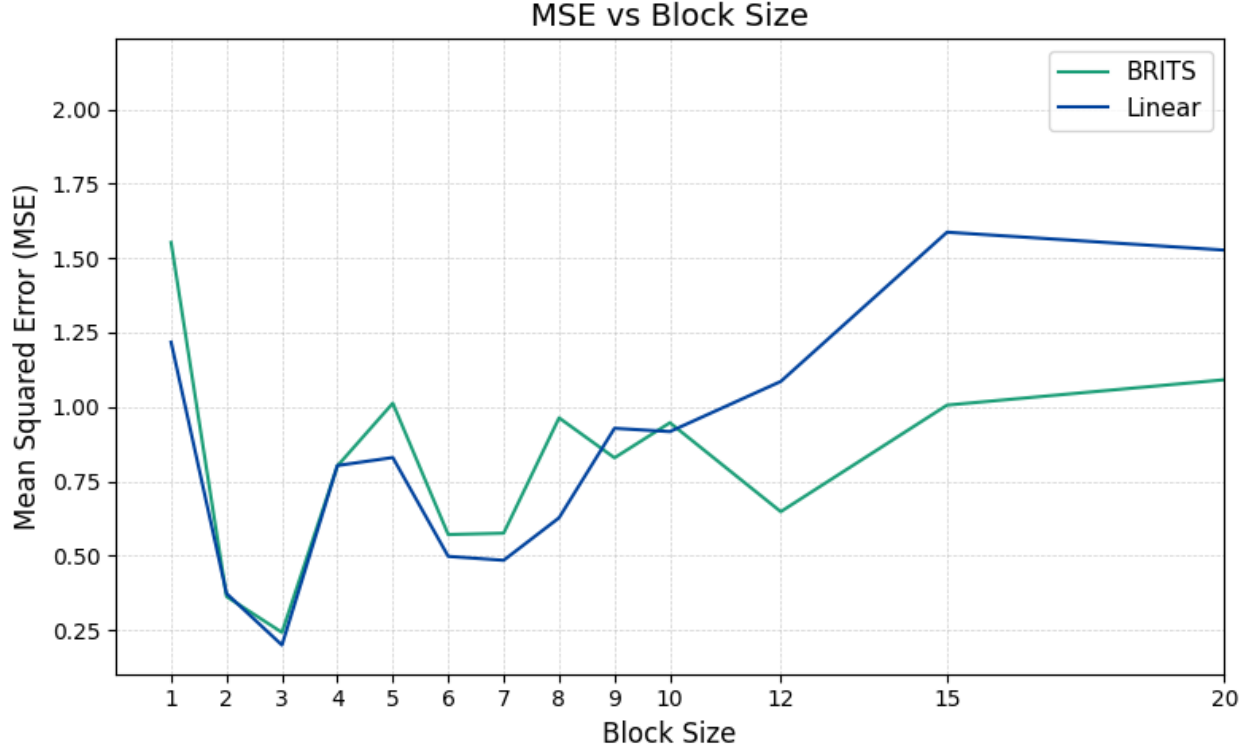


Figure 8: Zoom on Linear vs BRITS results from figure 6 for small block sizes

4.7 Final Imputation Strategy

Based on the evaluation results, the final imputation strategy adopted is a hybrid approach combining linear interpolation and Prophet [3]. Specifically, for small missing gaps (threshold set at 8 consecutive missing points, determined via cross-validation experiments figure 5), linear interpolation is applied due to its simplicity and high accuracy in these cases. For larger missing blocks exceeding this threshold, the Prophet [3] method is employed, leveraging its capacity to model seasonal patterns and provide stable imputations over extended sequences of missing data. (figure 9)

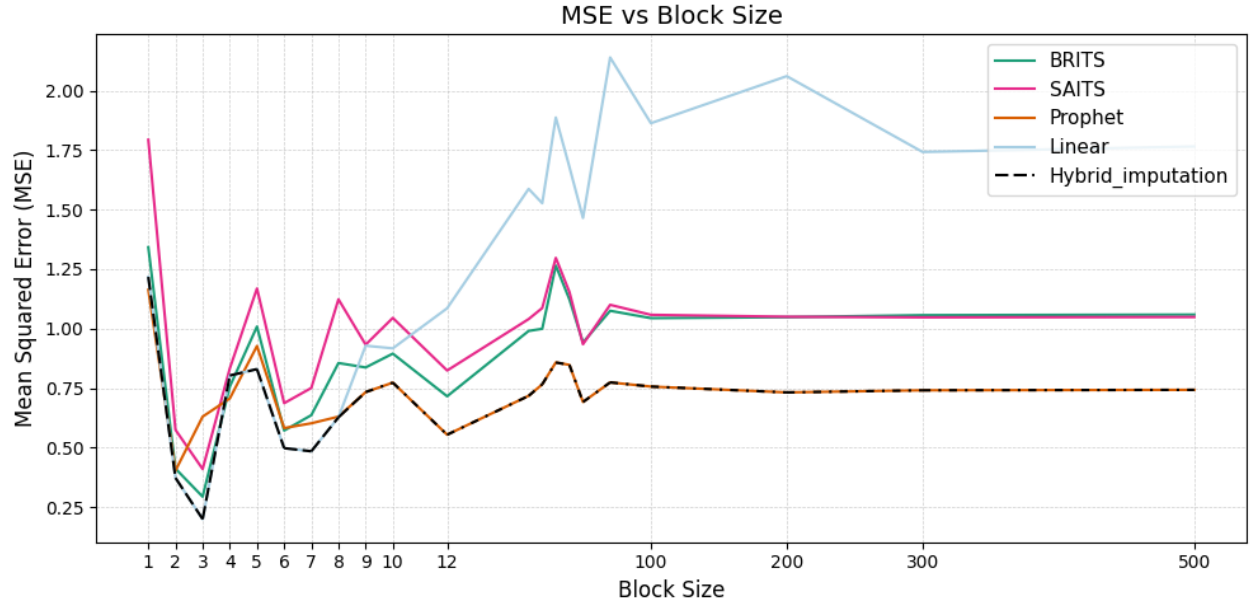


Figure 9: Performance of the hybrid-method (Linear + Prophet)

This concludes the first part of the internship, dedicated to the preprocessing and imputation of missing data within the water distribution network’s time series. Having established a robust and adaptive imputation strategy, the next phase of the work focuses on leveraging the structural information of the water distribution network. In particular, we aim to represent the network as a graph and apply graph-based inference techniques inspired by PhD thesis [1]. The objective is to perform flow estimation and prediction tasks directly on the graph representation of the network, integrating both topological and temporal information.

5 Graph-Based Inference on the Water Distribution Network

5.1 Introduction and Objectives

The second part of the internship focuses on leveraging the structure of the water distribution network to perform flow inference and prediction tasks. The main objective is to reconstruct the hydraulic network as a graph and implement methodologies inspired by [1], which proposed techniques for flow estimation using graph-based models.

Thus my task were to understand, implement, and apply the inference methods described in the thesis to our own reconstructed graph model of the Eau d’Azur water distribution network for the commune of Trinité.

5.2 Dumora’s Thesis overview

The thesis [1] proposes a methodology for inferring unmeasured quantities in potable water distribution networks using a graph-based approach. His thesis integrates network theory, flow algorithms, and machine learning-based signal inference to estimate unmeasured flows or demands across a network with limited sensor deployment. The approach consists of three key components: network graph construction, flow inference via Ford-Fulkerson [14] algorithm, and signal reconstruction using Kernel Ridge Regression.

Graph Construction The water distribution network is represented as a directed graph $G = (V, E)$, where V denotes the set of nodes (junctions, reservoirs, measurement points) and E the set of edges (pipes). Each edge has a capacity associated with the physical characteristics of the conduit (e.g., pipe diameter). The adjacency matrix A of the graph encodes connections between nodes, while the degree matrix D and Laplacian $L = D - A$ are used for spectral analysis. This structural representation is essential for applying both flow conservation principles and inference algorithms throughout the network.

Flow Inference using Ford-Fulkerson Algorithm The first inference step involves estimating flow distributions through the network using the Ford-Fulkerson [14] method for computing maximum flows. The method is iterative, working on the residual graph G_f where each edge’s residual capacity is updated after each augmentation step. Given partial flow measurements (from sensors) and physical constraints (e.g., directionality, capacity limits), the algorithm incrementally saturates the flows along augmenting paths that respect the observed data. Dumora adapts this approach to ensure that measured flows are used as constraints, leading to a constrained maximum flow solution rather than a purely theoretical maximum flow.

Signal Inference using Kernel Ridge Regression Once the global flow distribution is established, Dumora introduces a second step to infer unmeasured signals (e.g., chlore, flow rates) at some nodes of the graph. This is done through Kernel Ridge Regression (KRR), adapted to the graph structure. The Laplacian of the graph is used to construct a kernel matrix, enabling the propagation of observed signals to unmeasured nodes while enforcing smoothness across the network topology. The KRR formulation minimizes a trade-off between fitting the observed data and ensuring smoothness of the inferred signals across connected nodes, controlled by a regularization parameter λ .

The following sections detail how I reconstructed the Eau d’Azur network graph and re-implemented Dumora’s inference algorithms to apply them to our dataset.

5.3 Reconstruction of the Water Distribution Network as a Graph

Figure 1 illustrates the water distribution network under study. The objective is to transform this physical infrastructure into a graph structure, enabling a unified and analyzable representation of the network. Following the methodology described by [1], this transformation is natural since the water network inherently exhibits a graph topology.

In this representation, the pipes (conduites d’eau) are modeled as edges, capturing the physical connections between network components. Nodes are assigned to various elements such as meters (*compteurs*), represented in the Eau Azur database by attributes like *barchiera* and *cpt_764*. Each of these components corresponds to a node in the graph, allowing precise localization and labeling within the network topology.

The orientation of edges reflects the actual direction of water flow in the network, ensuring the graph structure remains consistent with the physical behavior of the system.

I implemented the graph construction using the `NetworkX`[15] Python package, which provides a flexible framework to create, manipulate, and analyze graph data structures.

By adopting this approach, it becomes straightforward to build a coherent data structure that aggregates information from multiple sources, facilitating the analysis of Eau d’Azur’s network through graph theory tools.

In the following steps, I proceed to the first inference method: the Ford-Fulkerson algorithm. Since this method operates on a graph structure, simply representing the entire network globally as a graph is not sufficient. To make the inference work, it is necessary to construct a specific type of graph that adheres to the algorithm’s requirements.

Thus, I needed to adjust the initial graph structure to ensure it aligns with the assumptions and constraints of the Ford-Fulkerson method. This involves modifying certain elements of the graph, such as defining clear source and sink nodes, ensuring correct edge capacities, and orienting the graph in a way that reflects potential flow paths.

The next section will present the detailed construction of this adapted graph structure, following the approach described by [1], and its integration with the inference method.

5.4 Reimplementation of the Flow-Based Inference Algorithm for Hydraulic Reconstruction of a Water Distribution Network

5.4.1 Transport Network and Basic Ford-Fulkerson Algorithm

A transport network is a directed graph where edges have capacities representing the maximum amount of flow that can traverse them. The classical Ford-Fulkerson algorithm computes the maximum flow from a single source node s to a single sink node t by repeatedly finding augmenting paths until no further improvements are possible.

5.4.2 Challenges of Water Distribution Networks

Water distribution networks, unlike standard transport networks, pose several unique challenges. They typically involve multiple water sources ($V_s \subset V$) and delivery points ($V_t \subset V$), creating a complex system of inputs and outputs. Flow directions in many pipes are not known in advance, resulting in an initially undirected network structure. Additionally, some nodes are equipped with sensors that provide real flow measurements, which impose constraints on the adjacent edges. Furthermore, each pipe is subject to physical capacity limits, with maximum flow determined by its diameter and a fixed maximum velocity.

5.4.3 Graph Adaptation Following Dumora’s Methodology

To apply a flow inference algorithm, Dumora adapted the network structure as follows:

Supersource and Supersink Construction A single supersource node s is connected to all source nodes $s_i \in V_s$ via edges (s, s_i) , each with a capacity equal to the measured inflow at s_i . Similarly, a supersink t is connected to all sink nodes $t_j \in V_t$ via edges (t_j, t) , with capacities matching the measured outflows.

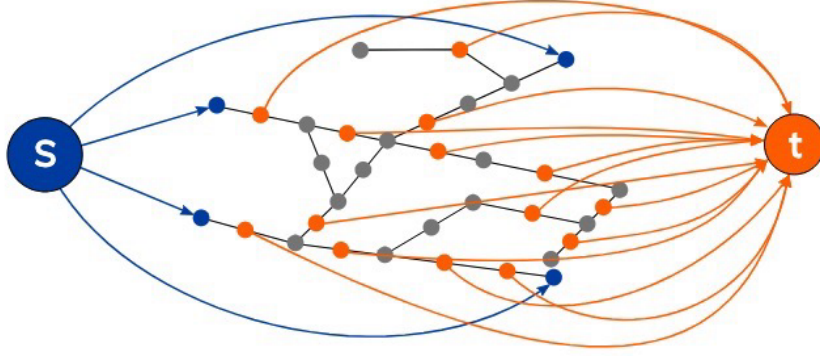


Figure 10: superSource and SuperSink construction, sources are in blue and sinks in red, others nodes and edges are represented in gray. Taken from [1]

Edge Capacities Based on Pipe Properties For each edge (u, v) , the capacity $C_{max}(u, v)$ is computed using:

$$C_{max}(u, v) = S_{u,v} \times V_{max}$$

where $S_{u,v} = \pi \times \left(\frac{d_{u,v}}{2}\right)^2$ is the cross-sectional area, and V_{max} is the fixed maximum flow velocity.

Handling Unknown Flow Directions Given the undirected nature of conduites, each edge is duplicated to create two antiparallel edges (u, v) and (v, u) , both initialized with the same capacity $C_{max}(u, v)$. However, water flow in a pipe cannot occur in both directions simultaneously. Thus, at any time, only one of the two edges may carry a non-zero flow, while the other becomes its residual arc.

Observed Flow Constraints For edges adjacent to nodes with sensors, the flow direction is fixed according to measurements. The capacities of these edges are set to the observed values, and flows through these edges are only allowed to increase to match observations. This introduces constraints ensuring that the algorithm respects the measured flows, even if the global network flow is not maximal.

5.5 Constrained Ford-Fulkerson Inference Method

The classical Ford-Fulkerson algorithm allows arbitrary choices in selecting augmenting paths from the source s to the sink t at each iteration. In [1], this path selection is constrained to prioritize arcs where flow observations exist, ensuring that these measured flows are satisfied as a priority.

Breadth-First Search (BFS) for Path Selection At every iteration, a simple path $p = \{s, v_1, \dots, v_n, t\}$ must be identified. To avoid exhaustive enumeration of all elementary paths, BFS is used to incrementally build a connected subgraph from s , marking nodes layer by layer. BFS ensures the discovery of shortest augmenting paths.

Constrained Augmenting Paths Through Observed Nodes A critical adaptation involves prioritizing the saturation of arcs adjacent to observed nodes V_{obs} where flow measurements (debitmètres) are available. These arcs $x_{obs} = \{(u, v) \mid u \in V_{obs}, v \in \Gamma(u)\}$ have known directions and flow rates. The algorithm ensures that, as long as there exists an unsaturated observed arc, the augmenting path must pass through it.

A bidirectional BFS is employed:

1. Search from s to an unsaturated observed node x_{obs} .
2. Then, search from x_{obs} to t .

The resulting path $p = s \rightarrow x_{obs} \rightarrow t$ must be elementary and acyclic. Only when all observed arcs are saturated does the algorithm revert to standard path selection from s to t .

Algorithm 1: Constrained Ford-Fulkerson Method

Input: $G = (V, E, c)$: transport network (V : nodes, E : edges, c : capacities)
 $G_f = (V, E_f, c_f)$: residual graph of G
Output: Maximal flow f from source s to sink t
while *augmenting path exists* **do**
 if *all observed nodes are unvisited or unsaturated* **then**
 $p_1 \leftarrow$ path from s to x_{obs}
 $p_2 \leftarrow$ path from x_{obs} to t
 $p \leftarrow p_1 \cup p_2$
 else
 $p \leftarrow$ path from s to t
 end
 $c_f(p) \leftarrow \min\{c_f(u, v) \mid (u, v) \in p\}$
 foreach *edge* $(u, v) \in p$ **do**
 $e_0 \leftarrow (u, v)$
 $e_1 \leftarrow (v, u)$
 if $f(u, v) > 0$ *and* $f(v, u) = 0$ **then**
 $f(u, v) \leftarrow f(u, v) + c_f(p)$
 else
 $f(v, u) \leftarrow f(v, u) + \max(0, c_f(p) - f(u, v))$
 $f(u, v) \leftarrow \max(0, f(u, v) - c_f(p))$
 end
 end
end

This constrained approach ensures that the observed flows are satisfied first, leading to a feasible hydraulic flow configuration that respects both physical constraints and sensor measurements. The final direction of the arcs is determined by the arcs carrying a strictly positive flow $f(u, v) > 0$.

5.6 Implementation and Validation

I implemented the constrained Ford-Fulkerson algorithm in Python using the **networkx** package to handle graph structures and operations. To ensure the correctness of the implementation, I validated the algorithm by running it on toy problems, replicating two test cases from [1].

The first test case is a simple flow problem without any constraints on directions or flow measurements (Figure 11), while the second included constraints on certain edges with fixed flow directions and measured flow values (Figure 12).

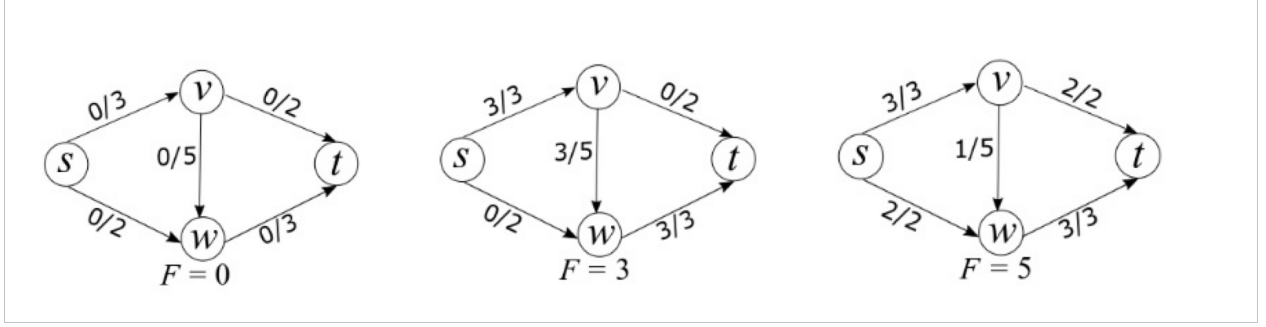


Figure 11: A flow network $G = (V, E)$ with a source s and a sink t . Each edge is labeled with its flow and capacity (flow/capacity). A maximum flow F in G with a total value of $F = 5$. Figure from [1]

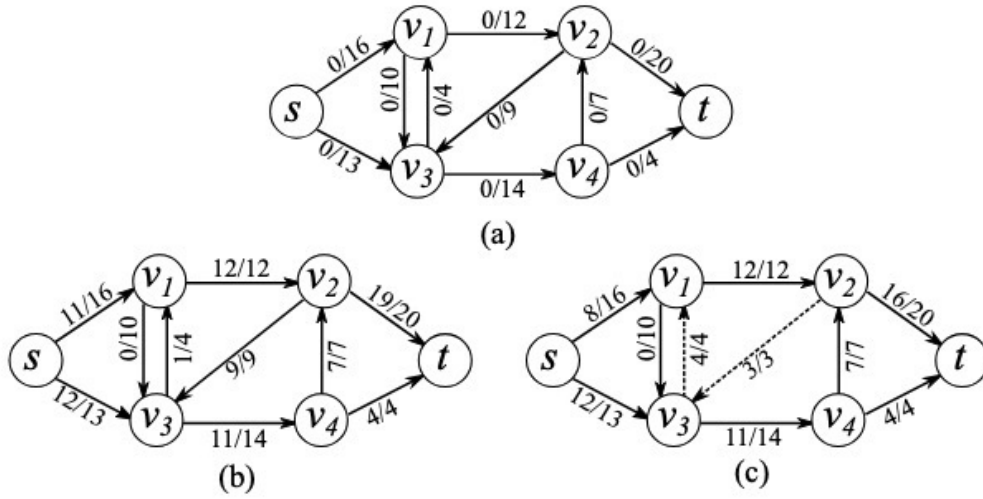


Figure 12: (a) A flow network $G = (V, E)$ with a source s and a sink t . Each edge is labeled with its flow and capacity (flow/capacity). (b) A maximum flow f_1 in G with a total value of $|f_1| = 23$. (c) A constrained flow f_2 in G with a total value of $|f_2| = 20$, respecting the direction and capacity constraints on the observed edges (v_3, v_1) and (v_2, v_3) . Figure from [1]

In both cases, the algorithm successfully reproduced the expected results, matching the solutions presented in the thesis. This confirmed the correctness of the implementation and its capacity to handle both unconstrained and constrained flow inference scenarios.

Following this validation, I applied the algorithm to the available data from the Eau d’Azur network. The dataset provided only partial information, corresponding to the right-hand section of the Trinité water distribution network (Figure 1).

For an initial test, I constructed a simplified subgraph in the shape of a Y (figure 13), starting from the source node `cpt_764` and extending to the nodes `SP GCMi (cpt_736)` and `Fuonsanta (cpt_772)`.

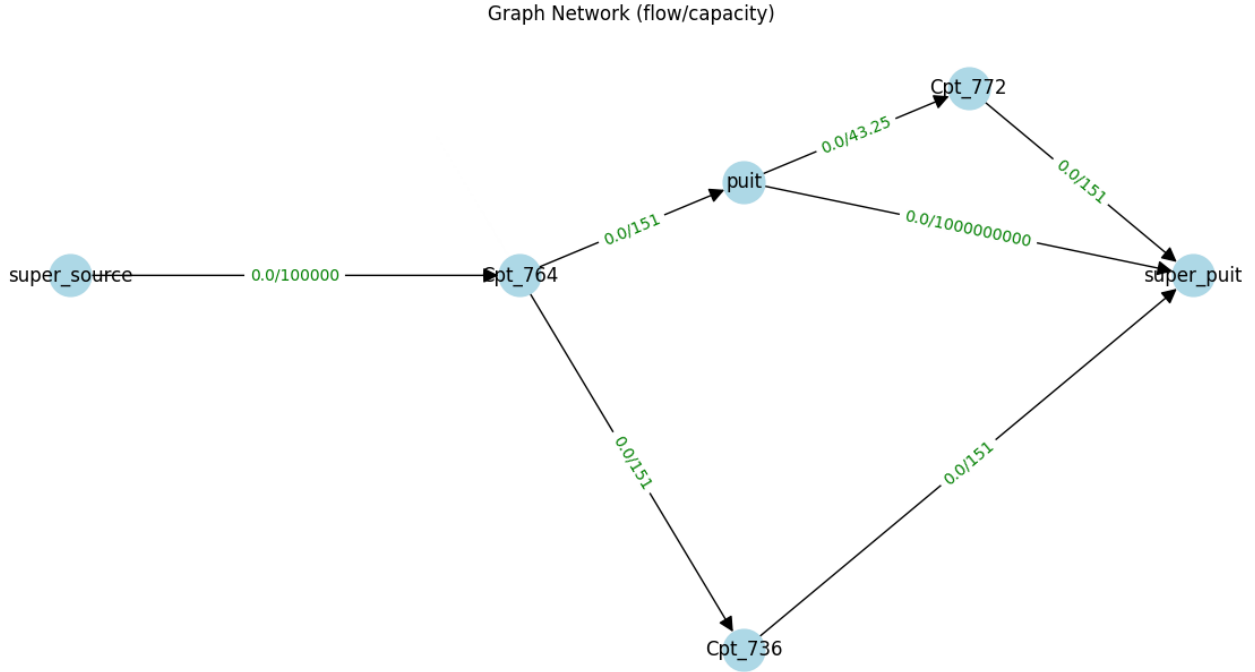


Figure 13: simplified subgraph of Trinité build in python with networkX package.

5.7 Limitations and Issues with Available Data

During the testing phase on our water network data, I encountered significant limitations regarding data quality and completeness, which affected the applicability of the constrained Ford-Fulkerson method. The flow data available for the sensors (`cpt_764`, `cpt_736`, `cpt_772`) were given as cumulative volumes in cubic meters (m^3), rather than instantaneous flow rates at each time step. Additionally, the dataset contained numerous missing values (NaNs) (25% for `cpt_764`) and we didn’t have any data available for the sensor `cpt_772`, making direct usage in the algorithm problematic in this subgraph and the full Trinité network.

Since the method requires instantaneous flow values at each exact time t , we computed these by taking the difference between consecutive cumulative values for all timestamps. However, the dataset has missing dates, so we inserted missing timestamps by adding 96 NaN entries per missing day. Thus taking the difference created anomalous values that distorted the flow calculations and introduced errors, complicating the application of the method.

Another critical issue arose when setting the capacity values for each edge. The method requires the diameter of the pipes and the maximum flow velocity to compute theoretical maximum capacities. However, lacking velocity measurements, we opted to approximate capacities using the maximum observed flow value for each sensor. This approach led to inconsistent and unrealistic results due to low volumes for one sensor. `Cpt_764`, the main source node, exhibited a maximum value of only 151 m^3 , while the others nodes `cpt_736` and `cpt_772` had maximums exceeding $40,000 \text{ m}^3$. This difference does not reflect the physical reality of the network and prevents meaningful validation of the method with this dataset.

5.8 Results of Constrained Ford-Fulkerson method on the subgraph

Despite these data limitations, I was still able to conduct preliminary experiments using the constructed graph and the implemented inference method. By selecting available data segments, I performed tests on the subgraph composed of cpt_764, cpt_736, and cpt_772. These experiments demonstrated that the constrained Ford-Fulkerson method operates correctly on this structure, with the flow propagating from the source cpt_764 to the sinks.

The algorithm successfully saturated the edge leading to cpt_772 with the expected flow. However, on the branch towards cpt_736, the flow reconstruction was problematic due to the presence of an intermediary sink node representing the client consumption block (referred to as the "boucle Trinité", shown as white squares in Figure 1). This intermediary node aggregates consumption for multiple clients, but we lack accurate measurements to define a correct capacity ($C_{\max}$) for its outgoing edge.

As a result, we were forced to assign an arbitrarily large capacity ($C_{\max} > 100000000$) to this edge leading to the super-sink, shown in Figure 13. Consequently, the flow is directly routed from this intermediary sink to the super-sink, preventing the algorithm from reconstructing the flow correctly at cpt_736, which lies downstream of this node. This structural issue limits the inference method's ability to propagate flow information beyond the intermediary sink in the absence of reliable consumption data.

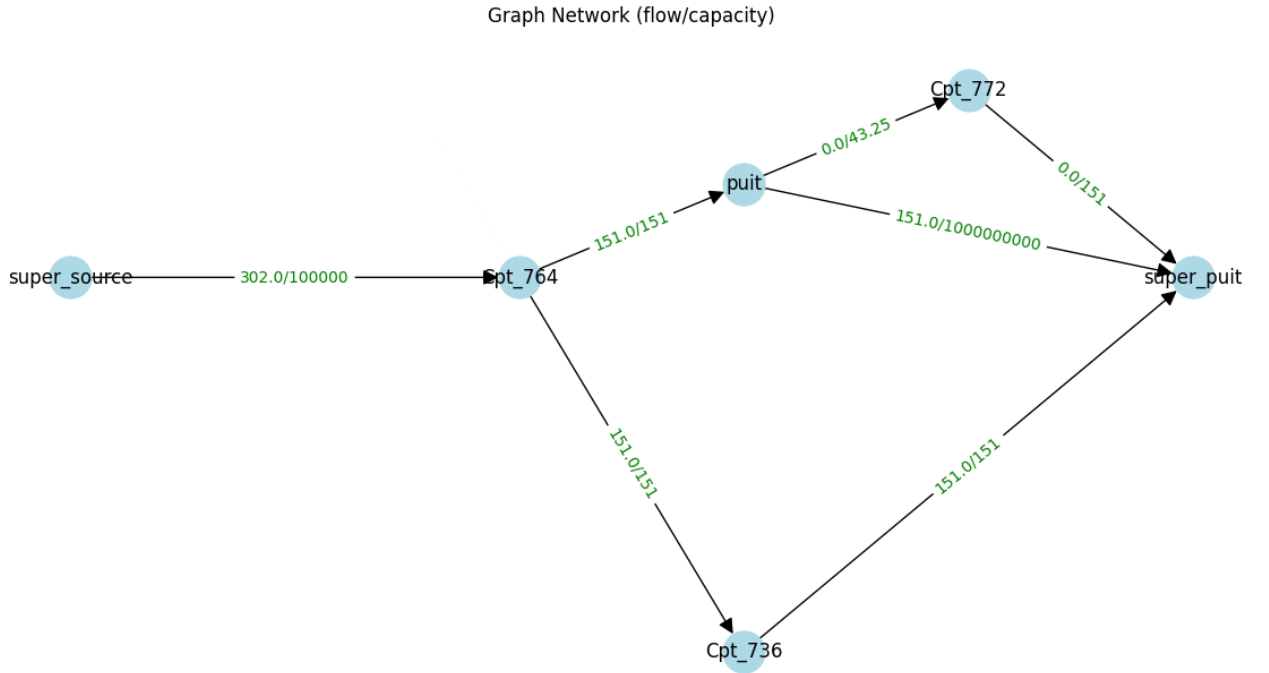


Figure 14: Results obtained on the subgraph with the constrained ford-fulkerson method.

In conclusion, the constrained Ford-Fulkerson inference method is a promising approach for reconstructing flow directions and values within water distribution networks. The experiments conducted on synthetic and partial real-world subgraphs demonstrated the algorithm's ability to respect observed constraints and reconstruct coherent flow patterns. However, due to the limitations of the available data during this internship, we were not able to fully validate the method on the complete Trinité network.

Additionally, the presence of intermediary consumption blocks introduces structural ambiguities in the reconstruction process. Without accurate capacity estimations for these intermediary sinks, the method logically routes flows directly from these nodes to the super-sink, bypassing downstream nodes like cpt_736. This behavior, while correct from the algorithm's optimization perspective, prevents a complete reconstruction of the flow across the entire network.

6 Kernel Ridge Regression

After completing the study and implementation of the constrained Ford-Fulkerson algorithm, I focused on the second major methodological contribution of the PhD thesis [1]: the application of Kernel Ridge Regression (KRR) for reconstructing partially observed signals on water distribution networks.

The objective of this part of the work is to explore an alternative approach for inferring unmeasured flows in large-scale networks, where only a limited number of nodes are equipped with sensors. The method aims to predict the flows on non-instrumented nodes using statistical regression techniques while explicitly incorporating the known topology of the network graph. This approach differs from the flow-based methods like Ford-Fulkerson by addressing the problem through a data-driven statistical framework.

To understand and correctly apply the KRR method, I undertook a thorough reading of Dumora’s thesis, especially the parts dedicated to the theoretical development of graph-based regression methods. In particular, the thesis builds extensively on the book [16], which became a primary reference during my study. This book provided the foundational concepts for understanding graph-based kernels, Laplacian operators, and the statistical framework necessary for applying KRR to network-structured data.

In the following sections, I will first present the mathematical formulation of the Kernel Ridge Regression method applied to graphs, explaining how the kernel is constructed from the graph Laplacian and how the regression problem is posed. I will then describe the algorithmic approach and computational strategies proposed in [1] to efficiently solve these problems on large-scale water distribution networks. Following this, I will detail the steps of my Python implementation of the method and finally, the tests and results obtained during this internship.

6.1 Kernel Ridge Regression on Graphs

We consider an undirected graph $G = (V, E)$ composed of N_v nodes and N_E edges. Each node $i \in V$ represents a measurement point (e.g., a sensor or flowmeter) in the water distribution network. Two nodes i and j are connected by an edge $\{i, j\} \in E$ if a physical pipe exists between them.

The adjacency matrix A of G is a symmetric matrix of size $N_v \times N_v$ defined as:

$$A_{ij} = \begin{cases} 1, & \text{if } \{i, j\} \in E \\ 0, & \text{otherwise} \end{cases}$$

The degree matrix D is a diagonal matrix where each diagonal entry d_i corresponds to the degree of node i , defined as the sum of weights of edges connected to i :

$$d_i = \sum_{j \neq i} A_{ij} \quad \text{thus} \quad D = \text{diag}(d_i)$$

The graph Laplacian is then given by:

$$L = D - A$$

Let $X = (X_1, \dots, X_{N_v})$ be a vector of real-valued node attributes (e.g., flow measurements). We assume observations are only available on a subset $V_{obs} \subset V$ of nodes, with observed values collected in the vector $x_{obs} = (X_i)_{i \in V_{obs}}$. Let $n = |V_{obs}|$ denote the number of observed nodes.

The objective is to predict the values of X on the unobserved nodes using a model $\hat{h} : V \rightarrow \mathbb{R}$, ensuring it aligns with the observed data while smoothly extending to the rest of the graph.

Definition of the Kernel Ridge Regression: Following [1] and [16], the problem is formulated as a ridge regression on graphs using a Laplacian-based kernel.

Given the graph Laplacian L , we define the kernel matrix K as its Moore-Penrose pseudo-inverse, $K = L^+$. Denoting Δ as the diagonal matrix of eigenvalues of K , and Φ as the matrix of corresponding eigenvectors, we can express functions over the graph in the spectral domain.

The Kernel Ridge Regression problem is then formulated as finding the estimator $\hat{\beta}$ that minimizes the following penalized least-squares criterion:

$$\hat{\beta} = \arg \min_{\beta \in \mathbb{R}^{N_v}} \sum_{i \in V_{obs}} (X_i - (\Phi\beta)_i)^2 + \lambda \beta^T \Delta_L \beta$$

where λ is the regularization parameter controlling the trade-off between data fitting and smoothness over the graph structure.

The final predictor $\hat{f}$ over all nodes is obtained by:

$$\hat{f} = \Phi \hat{\beta}$$

This formulation allows us to infer the signal on unobserved nodes by leveraging the graph structure, ensuring a smooth extension of the observed data across the network.

In the next sections, I will detail the algorithmic approach proposed by Dumora to efficiently solve this problem on large water distribution graphs, the challenges encountered in terms of computation and scalability, and how I implemented the method in Python during this internship.

Penalization Term and Graph Smoothness Hypothesis: Focusing on the penalization term in regression problem, we have:

$$\beta^T \Delta_L \beta = \beta^T \Phi^T \Phi \Delta_L \Phi \Phi^T \beta = f^T L f,$$

where $f = \Phi \beta$ is the predicted signal on the graph nodes.

This term can be expressed as:

$$f^T L f = \sum_{\{i,j\} \in E} (f_i - f_j)^2,$$

This expression clearly shows that the penalization enforces a smoothness constraint over the graph: it penalizes large differences in predicted values between neighboring nodes connected by an edge. In other words, the model inherently assumes that nodes close to each other (strongly connected) should have similar values.

Final Form of the Estimator The Phd thesis [1] derives an explicit form of the estimator by formulating the problem in matrix form using a masking matrix Ω to handle observed nodes. The optimization problem becomes equivalent to:

$$\hat{f} = \arg \min_{f \in \mathbb{R}^{N_v}} \|\Omega x - \Omega f\|_2^2 + \lambda f^T L f$$

Solving this yields the estimator:

$$\hat{f} = (M + \lambda L)^{-1} M x$$

where $M = \Omega^T \Omega$ is a diagonal matrix indicating observed nodes.

The estimator in the spectral basis is then given by:

$$\hat{\beta} = \Phi^T \hat{f} = \Phi^T (M + \lambda L)^{-1} M x$$

Resolution Methods: Three possible methods to solve the linear system derived from the ridge regression on graphs from the Phd thesis [1] Each method has its computational trade-offs depending on the graph size and observation density.

- **Method 1: Direct Inversion**

The solution is directly expressed as:

$$\hat{f} = (M + \lambda L)^{-1} M x$$

This method involves computing the inverse (or pseudo-inverse) of the matrix $(M + \lambda L)$, which is symmetric but potentially very large. For graphs with a high number of nodes, this becomes computationally infeasible due to memory and time constraints. Furthermore, computing $\hat{\beta} = \Phi^T \hat{f}$ would require diagonalizing L , which is impractical for large-scale graphs.

- **Method 2: Representer Theorem Approach**

This method leverages the representer theorem to express the solution in terms of kernel functions derived from the Moore-Penrose pseudo-inverse of the Laplacian:

$$\hat{f} = K_{N_v, n} \hat{\alpha} = K_{N_v, n} (K^{(n)} + \lambda I)^{-1} x_{obs}$$

This requires precomputing the kernel matrix $K = L^+$, which again involves computing the pseudo-inverse of L . If the number of observed nodes n is much smaller than N_v , the inversion of $(K^{(n)} + \lambda I)$ becomes tractable. However, this approach still suffers from scalability issues on large graphs due to the need to compute and store dense kernel matrices.

- **Method 3: Iterative Approach**

Using Conjugate Gradient Descent iteratively solve $(M + \lambda L)f = Mx$ using sparse matrix operations. This method is scalable for large graphs and avoids explicit inversion.

Algorithm 2: Conjugate Gradient Descent

```

Input:  $A = (M + \lambda L)$  with  $M = \Omega^T \Omega$ ,  $b = \Omega^T z$ 
Output:  $x_{k+1}$ 
Initialize  $x_0$  arbitrarily
 $r_0 := b - Ax_0$ 
 $p_0 := r_0$ 
 $k := 0$ 
while  $\|r_k\| > \epsilon$  do
     $\alpha_k := \frac{r_k^T r_k}{p_k^T A p_k}$  (step size)
     $x_{k+1} := x_k + \alpha_k p_k$  (next iterate)
     $r_{k+1} := r_k - \alpha_k A p_k$  (gradient update)
     $\beta_k := \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k}$  (conjugate parameter)
     $p_{k+1} := r_{k+1} + \beta_k p_k$  (descent direction update)
     $k := k + 1$ 
end

```

I chose to implement the method 3 with the iterative approach using the conjugate gradient descent method in Python.

6.2 Results and Limitations of the Method

I implemented the kernel ridge regression with the iterative method using the the conjugate gradient descent from scratch in Python. I built the full graph of the right upper part of the Trinité Network. All associated matrices, including the Laplacian, were constructed and manipulated using SciPy's [17] sparse linear algebra routines.

Regarding the regularization parameter λ , the original approach in [1] suggests performing cross-validation to select an optimal value based on the graph size and complexity. However, given the smaller scale of our network and the limitations issues with the data mentioned above, I decided to choose their optimal lambda found by cross-validation $\lambda = 0.05453$. The method was tested on the same two test cases as in previous sections to validate the correctness of the implementation.

An important observation from these tests concerns the core limitation of Laplacian-based penalization. The method enforces *smoothness* by constraining neighboring nodes to have similar values. However, it does not inherently respect physical conservation laws, such as Kirchhoff's law of flow conservation (i.e., input flow equals output flow at each node).

To demonstrate this limitation, I conducted a test on the simple toy problem (figure 11) where I masked the sink(p) and w nodes. The predicted values were as follows:

Node	Real	Predicted
s	5	5.00
v	2	2.00
w	3	3.20
p	5	2.60

Table 5: Test 1 — Prediction results with sink(p) and w nodes masked

As shown, the prediction at node p deviates significantly from its true value. This deviation is a direct consequence of the Laplacian smoothing term, which ensures that adjacent nodes have close values but does not enforce any flow conservation constraints. The model is purely driven by graph adjacency and the Laplacian, without accounting for water flow directionality, hydraulic constraints, or conservation laws inherent to the physical network.

This limitation is critical: the method cannot capture the hydraulic behaviors of the network since it lacks any notion of directed flows, sources, sinks, or flow conservation at junctions. Therefore, while the method is mathematically sound for general graph regression, it is unsuitable for our use case where respecting the hydraulic model is essential for accurate pressure and flow predictions.

7 Redefining the Regression Problem to Enforce Kirchhoff’s Law

After identifying the limitations of the regression method based on Laplacian penalization, particularly its inability to respect Kirchhoff’s law of flow conservation, the idea emerged to redesign the problem formulation. The goal is to construct a regression model on the graph that inherently enforces the physical flow conservation constraints of the hydraulic network.

To achieve this, we propose to reformulate the problem using the incidence matrix of the graph, which captures the relationship between nodes and edges in terms of flow directions.

Let $M \in \mathbb{R}^{N_v \times N_e}$ denote the incidence matrix, where N_v and N_e are the numbers of nodes and edges, respectively. Each column of M corresponds to an edge, with entries $+1$ and -1 indicating the edge’s orientation relative to its starting and ending nodes.

Let $d \in \mathbb{R}^{N_e}$ represent the vector of edge flows, and $f = Md \in \mathbb{R}^{N_v}$ denote the vector of node divergences, representing the net inflow minus outflow at each node. Kirchhoff’s law of flow conservation is thus enforced by the linear relation

$$f = Md \quad (1)$$

Given partial observations of node divergences f_{obs} at a subset of nodes $V_{\text{obs}} \subset V$, the objective is to infer the edge flows d and, implicitly, the divergences at unobserved nodes $f_{\text{miss}} = f_{V \setminus V_{\text{obs}}}$. To formalize this, a selection matrix $\Omega_f \in \mathbb{R}^{|V_{\text{obs}}| \times N_v}$ is introduced to extract the observed entries from the full vector f . The regression problem is then formulated as minimizing the squared error:

$$\min_{d \in \mathbb{R}^{N_e}} \|\Omega_f(Md) - f_{\text{obs}}\|^2 \quad (2)$$

This formulation ensures that Kirchhoff’s law holds globally by construction, the predicted divergences at observed nodes match the measured values f_{obs} in a least-squares sense, and the flows at unobserved nodes are implicitly predicted through the incidence matrix relation.

Current Status and Next Steps: As of the end of July, the internship is at the stage of finalizing this report for submission. The development of the new regression approach based on the incidence matrix formulation is scheduled for the last month of the internship, in August. The objective for August is to: design and implement an algorithm to solve the constrained regression problem enforcing Kirchhoff’s law, test the method on small, controlled cases to validate its correctness and extend the approach to larger subgraphs representative of real network data.

Since this work is planned for August, no implementation or results are available at this time. This task represents the main perspective and objective for the final phase of the internship.

8 Conclusion

The objective of this internship was to address the challenge of missing data in water distribution networks and to develop inference techniques for estimating unmeasured flows using graph-based methods. The first phase of the work focused on missing data imputation. After an in-depth analysis of the data structure and missing patterns, I implemented a robust evaluation framework, allowing the comparison of various time series imputation methods. Through this, we identified a hybrid strategy combining linear interpolation for small gaps and Prophet [3] for larger blocks as the most effective approach for our datasets.

In the second phase, I reconstructed the water distribution network as a directed graph and implemented inference methodologies inspired by the Phd thesis [1]. This involved building the graph structure from available data, applying flow inference techniques using adaptations of the Ford-Fulkerson [14] algorithm, and implementing signal propagation through Kernel Ridge Regression on the network graph. These methods provided a framework for estimating unmeasured flows and demands based on sparse sensor data, leveraging the physical and topological properties of the network. Although we could not fully validate the constrained Ford-Fulkerson algorithm due to the data limitations, the results demonstrate its as a promise flow inference method. Lastly for Kernel Ridge Regression, we observed that it did not conform to Kirchhoff's law, which motivated us to design and implement a new regression method that enforces this physical constraint. This new method aims to improve flow estimation at each node in the network and will be the focus of further work of the internship.

References

- [1] C. Dumora, “Estimation de paramètres clés liés à la gestion d’un réseau de distribution d’eau potable : Méthode d’inférence sur les noeuds d’un graphe,” Thèse de doctorat, Université de Bordeaux, 2020, statistiques [math.ST], NNT: 2020BORD0325, tel-03132438.
- [2] H. M. Manzano, “Modélisation du réseau de distribution d’eau: Méthodes d’interpolation et de prévision pour les comportements saisonniers,” Université Côte d’Azur, Rapport de stage (M1), July 2024.
- [3] S. J. Taylor and B. Letham, “Forecasting at scale,” PeerJ Inc., Tech. Rep. e3190v2, Sep. 2017, iSSN: 2167-9843. [Online]. Available: <https://peerj.com/preprints/3190>
- [4] O. Troyanskaya, M. Cantor, G. Sherlock, P. Brown, T. Hastie, R. Tibshirani, D. Botstein, and R. B. Altman, “Missing value estimation methods for dna microarrays,” *Bioinformatics*, vol. 17, no. 6, pp. 520–525, 2001.
- [5] A. Trindade, “Electricity,” <https://doi.org/10.24432/C58C86>, 2015, uCI Machine Learning Repository.
- [6] S. Chen, “Beijing multi-site air quality dataset,” <https://doi.org/10.24432/C5RK5G>, 2017, uCI Machine Learning Repository.
- [7] I. Pratama, A. E. Permanasari, I. Ardiyanto, and R. Indrayani, “A review of missing values handling methods on time-series data,” in *2016 International Conference on Information Technology Systems and Innovation (ICITSI)*. Bandung – Bali, Indonesia: IEEE, Oct. 2016.
- [8] W. Du, J. Wang, L. Qian, Y. Yang, Z. Ibrahim, F. Liu, Z. Wang, H. Liu, Z. Zhao, Y. Zhou, W. Wang, K. Ding, Y. Liang, B. A. Prakash, and Q. Wen, “TSI-Bench: Benchmarking Time Series Imputation,” Oct. 2024, arXiv:2406.12747 [cs]. [Online]. Available: <http://arxiv.org/abs/2406.12747>
- [9] J. Wang, W. Du, Y. Yang, L. Qian, W. Cao, K. Zhang, W. Wang, Y. Liang, and Q. Wen, “Deep Learning for Multivariate Time Series Imputation: A Survey,” May 2025, arXiv:2402.04059 [cs]. [Online]. Available: <http://arxiv.org/abs/2402.04059>
- [10] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Transactions of the American Society of Mechanical Engineers, Series D: Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [11] W. Cao, D. Wang, J. Li, H. Zhou, L. Li, and Y. Li, “BRITS: Bidirectional Recurrent Imputation for Time Series.”
- [12] W. Du, D. Côté, and Y. Liu, “SAITS: Self-attention-based imputation for time series,” *Expert Systems with Applications*, vol. 219, p. 119619, Jun. 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957417423001203>
- [13] W. Du, “PyPOTS: a Python toolbox for data mining on Partially-Observed Time Series,” *arXiv preprint arXiv:2305.18811*, 2023.
- [14] L. R. Ford and D. R. Fulkerson, “Maximal flow through a network,” *Canadian Journal of Mathematics*, vol. 8, pp. 399–404, 1956.
- [15] A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring network structure, dynamics, and function using networkx,” *Proceedings of the 7th Python in Science Conference (SciPy2008)*, pp. 11–15, 2008.
- [16] E. D. Kolaczyk and G. Csárdi, *Statistical Analysis of Network Data with R*, ser. Use R! Springer, 2014.
- [17] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, Í. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors, “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.